

BaCoN - Tutorial

Thomas Rohde

April 29, 2025

This vignette describes how to predict functional buffering between gene pairs using BaCoN, using data from the Cancer Dependency Map (DepMap).

0. Install BaCoN package

```
if(!require(devtools)) {install.packages("devtools")}  
if (F) {devtools::install_github("billmannlab/BaCoN", force = T)}
```

1. Import packages

```
library(BaCoN)  
library(data.table)  
library(stringr)  
library(magrittr)
```

2. Data preparation

This script depends on DepMap gene expression data as well as Chronos scores, available at the DepMap portal.

Please download the files and place them in the “data” directory.

- Chronos scores: “CRISPRGeneEffect.csv”
- Gene expression: “OmicsExpressionProteinCodingGenesTPMLogp1.csv”

2.1 Import DepMap gene expression and fitness effect data

```
import_depmap <- \(filepath) {  
  object <- as.matrix(data.table::fread(filepath), rownames = 1)  
  colnames(object) <- stringr::str_split_i(colnames(object), " \\(", 1)  
  return(object)  
}  
  
ge_complete <- import_depmap(  
  file.path("../", "data", "OmicsExpressionProteinCodingGenesTPMLogp1.csv"))  
  
chronos_complete <- import_depmap(  
  file.path("../", "data", "CRISPRGeneEffect.csv"))  
  
message(paste0(round(sum(is.na(chronos_complete)) /  
  length(chronos_complete) * 100, 2),
```

```

      " percent of values are imputed."))
#> 0.85 percent of values are imputed.

# Impute missing Chronos scores as gene-wise means

for (i in 1:ncol(chronos_complete)) {
  chronos_complete[,i][is.na(chronos_complete[,i])] <- mean(chronos_complete[,i], na.rm = T)
}

```

2.2 Subset expression and fitness effect matrix

The gene expression and the Chronos score matrix are reduced to the cell lines that are represented in both datasets.

```

intersecting_cell_lines <- intersect(rownames(ge_complete), rownames(chronos_complete))

ge_subset <- ge_complete[intersecting_cell_lines,]
chronos_subset <- chronos_complete[intersecting_cell_lines,]

```

The correlations are computed across 1103 intersecting cell lines.

Conceptually, we expect that biologically meaningful buffering predictions require the buffering partner to be expressed in a sufficient number of cell lines and the knockout of the buffered partner to have a certain impact on cellular fitness. Our experience showed that removing genes with low signal by applying a threshold on the expression level improves buffering predictions.

This is only one possible strategy. In contrast to correlation scores, BaCoN scores are compared with and thus influenced by all other gene partners of the respective pair. It is recommended to experiment with different matrix sizes.

To keep runtimes low, we apply very strict thresholds in this vignette. In a comprehensive analysis, it is strongly recommended to cover a larger fraction of the genome, by defining less strict thresholds (e.g. between 0.25 - 1). Matrix sizes of ~ 6000-16000 buffering as well as roughly the same number of buffered genes showed good results in our analyses. The cell lines should be kept at as high as possible.

```

# We restrict the gene space to genes that do not show an expression (log2 TPM+1)
# of > 5 in at least 1000 of the cell lines.

expression_genes <- names(which(apply(ge_subset >= 5, 2, sum, na.rm = T) >= 1000))

ge_subset <- ge_subset[,expression_genes]

chronos_genes <- names(which(apply(abs(chronos_subset) > 1, 2, sum) >= 30))
chronos_subset <- chronos_subset[,chronos_genes]

```

The resulting universe is reduced to 1460 expression genes and 1768 fitness genes.

```

prediction_matrices <- array(
  data = NA,
  dim = c(ncol(ge_subset), ncol(chronos_subset), 4),
  dimnames = list(colnames(ge_subset),
                   colnames(chronos_subset),
                   c("PCC", "BaCoN", "Cholesky_PCC", "Cholesky_BaCoN")))

```

3.1 Compute PCC correlation matrix:

```
prediction_matrices[,,"PCC"] <- cor(ge_subset,
                                   chronos_subset,
                                   use = "pairwise.complete.obs")
```

3.2 Compute BaCoN matrix:

BaCoN may behave unexpectedly if the input correlation matrix is formatted in an unexpected way. To ensure stable and reliable results, please ensure that the input matrix...

1. is a correlation matrix or composed of comparable values scaling between -1 and 1. BaCoN is currently not designed to work e.g. on negative logarithmic p-values or other association indices.
2. does not contain NAs. This should not cause problems in cases of single NA values, but entire NA rows or columns should be removed from the matrix.
3. should not have duplicated row- or column names. Each gene should be represented in one row and/or column only.

```
prediction_matrices[,,"BaCoN"] <- BaCoN(prediction_matrices[,,"PCC"])
```

4. Cholesky whitening + PCC + BaCoN

BaCoN alone creates a buffering prediction matrix that can be used, but we observed that combining BaCoN with pre-hoc covariance normalization methods further increases the quality of predictions (less false positives, higher number of paralogs). Next, we combine BaCoN with the pre-hoc covariance normalization method Cholesky whitening, as proposed in our manuscript.

We use the `whitening` package and its `whiten` function.

```
if (F) {install.packages(whitening)}
library(whitening)
#> Lade nötiges Paket: corpcor
```

We whiten both input matrices independently. It is important that the cell lines of both objects are identical and in the second dimension (columns) of the object. Therefore, we transpose the objects before and retranspose them after the whitening.

```
whitened_ge_subset <- t(whiten(t(ge_subset), method = "Cholesky"))
whitened_chronos_subset <- t(whiten(t(chronos_subset), method = "Cholesky"))

prediction_matrices[,,"Cholesky_PCC"] <- cor(
  whitened_ge_subset,
  whitened_chronos_subset,
  use = "pairwise.complete.obs")

prediction_matrices[,,"Cholesky_BaCoN"] <- BaCoN(prediction_matrices[,,"Cholesky_PCC"])
```

5. Collect top 1000 predictions:

We use the lowest BaCoN score of the top 1000 predictions as cutoff. The high density of paralog pairs is clearly visible. This can lead to more than 1000 pairs in the prediction set, as the top BaCoN-scored pairs are often tied. We use the PCC score to deal with the tied pairs.

```
predictions <- data.table(data.frame(
  expression_gene = rep(rownames(prediction_matrices),
                        ncol(prediction_matrices)),
```

```

chronos_gene = rep(colnames(prediction_matrices),
  each = nrow(prediction_matrices)),
  PCC = as.vector(prediction_matrices[,,"PCC"]),
  BaCoN = as.vector(prediction_matrices[,,"BaCoN"]),
  Cholesky_PCC = as.vector(prediction_matrices[,,"Cholesky_PCC"]),
  Cholesky_BaCoN = as.vector(prediction_matrices[,,"Cholesky_BaCoN"])
)
)

predictions[order(BaCoN, PCC, decreasing = T), BaCoN_rank := .I]
predictions[order(Cholesky_BaCoN, PCC, decreasing = T), Cholesky_BaCoN_rank := .I]

predictions <- predictions[order(Cholesky_BaCoN_rank, decreasing = F)]

predictions[1:25, .(expression_gene, chronos_gene, PCC, Cholesky_BaCoN)]
#>      expression_gene chronos_gene      PCC Cholesky_BaCoN
#>      <char>          <char>          <num>      <num>
#>  1:      TIMM17B      TIMM17A  0.402796282  0.9993804
#>  2:      CHMP4A      CHMP4B  0.356270247  0.9993804
#>  3:      PABPC4      PABPC1  0.351385289  0.9993804
#>  4:      UBE2C      UBE2S  0.217317198  0.9993804
#>  5:      DSTN      CFL1  0.503833765  0.9990706
#>  6:      SARAFA      PPP2CA  0.285569734  0.9990706
#>  7:      ACTG1      ACTB  0.203234554  0.9941140
#>  8:      RPS2      PDCD2  0.230229090  0.9938042
#>  9:      CHCHD2      MSTO1  0.153289129  0.9925651
#> 10:      DDX17      DDX5  0.242464728  0.9913259
#> 11:      CCT6A      MSTO1  0.160806827  0.9903965
#> 12:      KHDRBS1      MFN2 -0.056669189  0.9897770
#> 13:      EIF4A3      IPO11  0.161218525  0.9894672
#> 14:      TUBB      TUBB4B  0.375054473  0.9888476
#> 15:      DDX39A      DDX39B  0.324062472  0.9885378
#> 16:      MRPS21      MRPS21  0.023700158  0.9845105
#> 17:      ZBTB80S      DHX36  0.076016285  0.9835812
#> 18:      RAC1      RBBP6  0.011465075  0.9820322
#> 19:      CHMP2A      CHMP3  0.330602239  0.9792441
#> 20:      ETVB      ADSL  0.069429072  0.9792441
#> 21:      SBDS      ZNF407  0.047587250  0.9767658
#> 22:      NOP10      PSMG1  0.029436062  0.9752169
#> 23:      ATP5F1E      RPP14  0.066836447  0.9745973
#> 24:      SLBP      SERBP1  0.006998759  0.9742875
#> 25:      TXNL4A      VPS4A  0.343127157  0.9736679
#>      expression_gene chronos_gene      PCC Cholesky_BaCoN

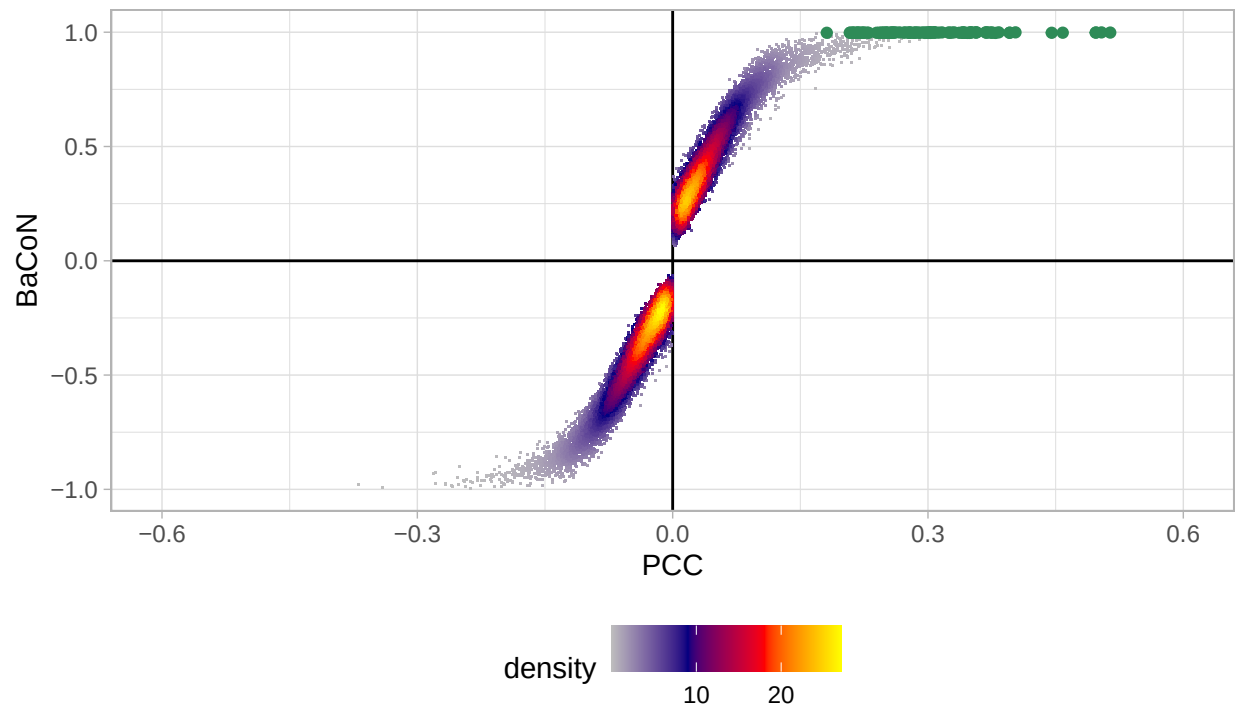
```

6. Show score distribution and predictions

```
#> Warning: Paket 'ggplot2' wurde unter R Version 4.4.3 erstellt
```

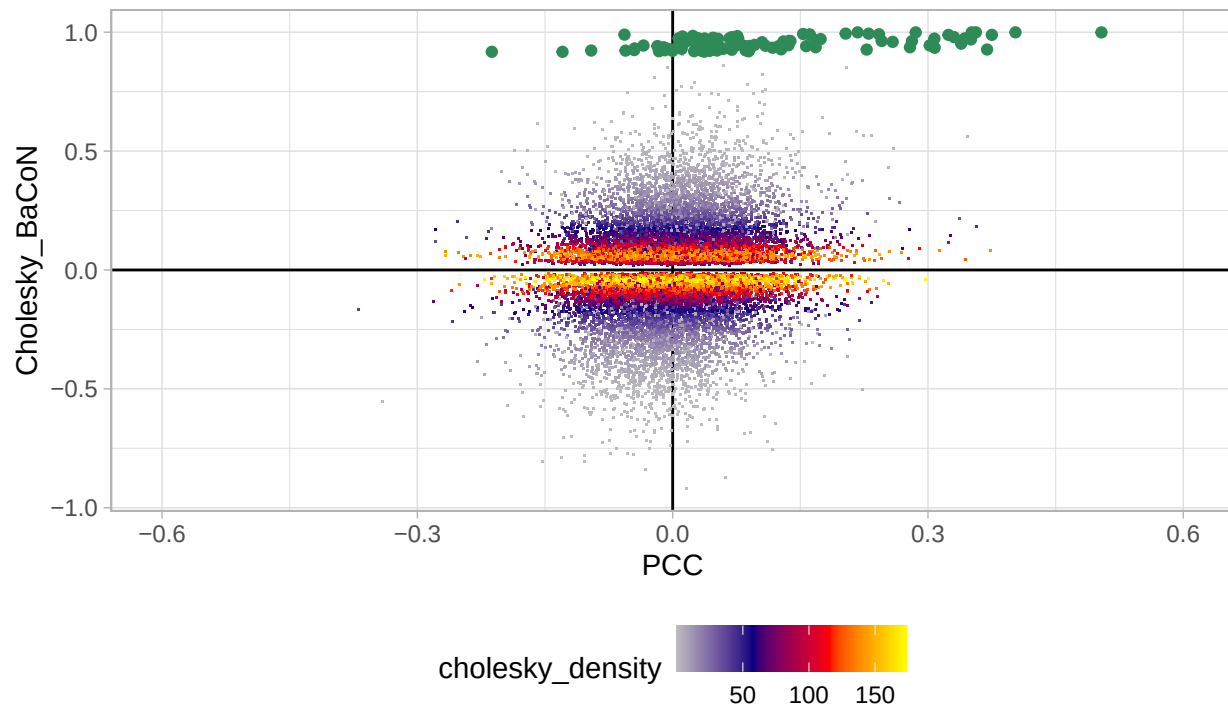
BaCoN vs PCC scores

Heatscatter plots of BaCoN ~ PCC coordinates of gene pairs.
Gene pairs with the top 100 BaCoN scores are colored green.



Cholesky + BaCoN vs PCC scores

Heatscatter plots of Cholesky+BaCoN ~ PCC coordinates of gene pairs.
Gene pairs with the top 100 Cholesky+BaCoN scores are colored green.



Summary

```
str(ge_complete)
#>  num [1:1673, 1:19193] 4.33 4.57 3.15 4.24 4.03 ...
#>  - attr(*, "dimnames")=List of 2
#>    ..$ : chr [1:1673] "ACH-001113" "ACH-001289" "ACH-001339" "ACH-001979" ...
#>    ..$ : chr [1:19193] "TSPAN6" "TNMD" "DPM1" "SCYL3" ...

str(chronos_complete)
#>  num [1:1178, 1:17916] -0.122 -0.0165 -0.1848 -0.0719 -0.0192 ...
#>  - attr(*, "dimnames")=List of 2
#>    ..$ : chr [1:1178] "ACH-000001" "ACH-000004" "ACH-000005" "ACH-000007" ...
#>    ..$ : chr [1:17916] "A1BG" "A1CF" "A2M" "A2ML1" ...

str(ge_subset)
#>  num [1:1103, 1:1460] 7.11 7.15 6.54 6.09 6.53 ...
#>  - attr(*, "dimnames")=List of 2
#>    ..$ : chr [1:1103] "ACH-001289" "ACH-001538" "ACH-000242" "ACH-000233" ...
#>    ..$ : chr [1:1460] "DPM1" "LASP1" "M6PR" "ARF5" ...

str(chronos_subset)
#>  num [1:1103, 1:1768] -1.24 -0.869 -0.83 -0.845 -1.109 ...
#>  - attr(*, "dimnames")=List of 2
#>    ..$ : chr [1:1103] "ACH-001289" "ACH-001538" "ACH-000242" "ACH-000233" ...
```

```

#> ..$ : chr [1:1768] "AAMP" "AARS1" "AARS2" "AATF" ...

str(prediction_matrices)
#> num [1:1460, 1:1768, 1:4] 0.07661 0.10543 0.00668 0.08391 0.01036 ...
#> - attr(*, "dimnames")=List of 3
#> ..$ : chr [1:1460] "DPM1" "LASP1" "M6PR" "ARF5" ...
#> ..$ : chr [1:1768] "AAMP" "AARS1" "AARS2" "AATF" ...
#> ..$ : chr [1:4] "PCC" "BaCoN" "Cholesky_PCC" "Cholesky_BaCoN"

str(predictions)
#> Classes 'data.table' and 'data.frame': 2581280 obs. of 8 variables:
#> $ expression_gene : chr "TIMM17B" "CHMP4A" "PABPC4" "UBE2C" ...
#> $ chronos_gene : chr "TIMM17A" "CHMP4B" "PABPC1" "UBE2S" ...
#> $ PCC : num 0.403 0.356 0.351 0.217 0.504 ...
#> $ BaCoN : num 0.999 0.999 0.999 0.999 0.999 ...
#> $ Cholesky_PCC : num 0.153 0.153 0.164 0.178 0.142 ...
#> $ Cholesky_BaCoN : num 0.999 0.999 0.999 0.999 0.999 ...
#> $ BaCoN_rank : int 1 17 5 26 15 11 2062 90 1163 105 ...
#> $ Cholesky_BaCoN_rank: int 1 2 3 4 5 6 7 8 9 10 ...
#> - attr(*, ".internal.selfref")=<externalptr>

```

Sessioninfo

```

sessionInfo()
#> R version 4.4.2 (2024-10-31 ucrt)
#> Platform: x86_64-w64-mingw32/x64
#> Running under: Windows 11 x64 (build 26100)
#>
#> Matrix products: default
#>
#> locale:
#> [1] LC_COLLATE=German_Germany.utf8 LC_CTYPE=German_Germany.utf8
#> [3] LC_MONETARY=German_Germany.utf8 LC_NUMERIC=C
#> [5] LC_TIME=German_Germany.utf8
#>
#> time zone: Europe/Berlin
#> tzcode source: internal
#>
#> attached base packages:
#> [1] stats graphics grDevices utils datasets methods base
#>
#> other attached packages:
#> [1] ggplot2_3.5.2 whitening_1.4.0 corpcor_1.6.10 magrittr_2.0.3
#> [5] stringr_1.5.1 data.table_1.17.0 BaCoN_0.0.0.9000
#>
#> loaded via a namespace (and not attached):
#> [1] gtable_0.3.6 dplyr_1.1.4 compiler_4.4.2 crayon_1.5.3
#> [5] tidyselect_1.2.1 tinytex_0.56 progress_1.2.3 scales_1.3.0
#> [9] yaml_2.3.10 fastmap_1.2.0 R6_2.6.1 labeling_0.4.3
#> [13] generics_0.1.3 knitr_1.50 MASS_7.3-65 tibble_3.2.1
#> [17] munsell_0.5.1 pillar_1.10.2 rlang_1.1.5 stringi_1.8.7

```

```
#> [21] xfun_0.51          cli_3.6.4          withr_3.0.2        digest_0.6.37  
#> [25] grid_4.4.2         rstudioapi_0.17.1 hms_1.1.3          lifecycle_1.0.4  
#> [29] prettyunits_1.2.0 vctrs_0.6.5        evaluate_1.0.3     glue_1.8.0  
#> [33] farver_2.1.2       job_0.3.1          colorspace_2.1-1  rmarkdown_2.29  
#> [37] tools_4.4.2        pkgconfig_2.0.3    htmltools_0.5.8.1
```